An Agentic Architecture for Embodied Mathematics

Nazmus Saquib *[1,2], Razin Reaz Abedin[1], Tahshina Islam Munni[1], and Mustafa Siam-ur-Rafique[1]

[1]Noyon PBC, San Mateo, California, USA
[2]Oak Network Inc., USA

Abstract

Scientific and Mathematical Computing practitioners rely on abstract symbolic and numeric computation for their typical workflows. On the other hand, science and math educators aim to explain abstract mathematical concepts using embodied, metaphorical representations when possible. Visual grammars and interactions developed in many educational and prototypical interactions often do not scale to a wide variety of abstractions typical in mathematics. However, with the advent of LLM-powered agents in the recent years, we believe embodied and agentic interactions for doing mathematics will take on new directions. In this paper, we present a roadmap for agent-assisted embodied interactions for scientific and mathematical computing, inspired by the architectures of coding assistants and previous research in the HCI community. Three separate systems define our architecture: an LLM-assisted visual grammar agent to extract mathematical relationships and elements from text, a set of coding agents crafted to notate drawings and handle embodied interactions in coding IDEs, and an enhanced tool calling pipeline that utilizes dynamic code generation to work in unison with the two systems above. We envision enabling a wide range of applications both in education and professional scientific computing via this architecture. We invite the scientific and HCI communities to comment on this direction.

Keywords: Agentic AI, Mathematics, Scientific Computing, Human-Computer Interaction.

1. Introduction

"*I now view the effort to find a fundamental theory of physics as in many ways another challenge in language design — perhaps even the ultimate such challenge. Designing a computational language is to create a bridge between two domains: the abstract world of what is possible to do, and the "mental" world of what people understand and are interested in doing. The challenge is [...] to give people a way to describe these [abstractions].*" - Stephen Wolfram [31]

Mathematics as a discipline has relied on symbolic abstractions and symbol manipulations for the last few centuries. The power and hierarchical flexibility of symbols to do mathematics at higher dimensions and dealing with abstract concepts is undeniable. Algebra has been widely used to manipulate symbols for abstract mathematics. However, such abstraction is not always intuitive, and creates a barrier to explanation, exploration, and understanding [29].

In practice, many learners lose the natural, intuitive underpinnings of those abstractions and resort to memorizing rules of manipulation ("symbol pushing") which become cognitively arbitrary and severed from their intuitive grounding [29]. Current Computer Algebra System (CAS) software (such as [20, 14, 19, 21, 8]) only provide notebook interfaces where code and plots can be combined, which themselves are not embodied mathematics, and requires working with abstractions right away. Power users of symbols and code enjoy the constructs afforded by abstract algebra and computational tools.

In a world that is increasingly moving towards natural, intuitive computing, both learners and practitioners deserve interfaces that harness the power of symbols and abstraction but keep the option for natural, intuitive interactions open [26]. Implementing custom interfaces for every use case in mathematics, science, and engineering is cumbersome and unscalable. In practice, learners and practitioners fall back to established workflows because they are not interface builders, and interface builders cannot possibly build for all kinds of mathematical abstractions and tools. The design space is rich and complex.

*Corresponding author: saquib@noyon.ai

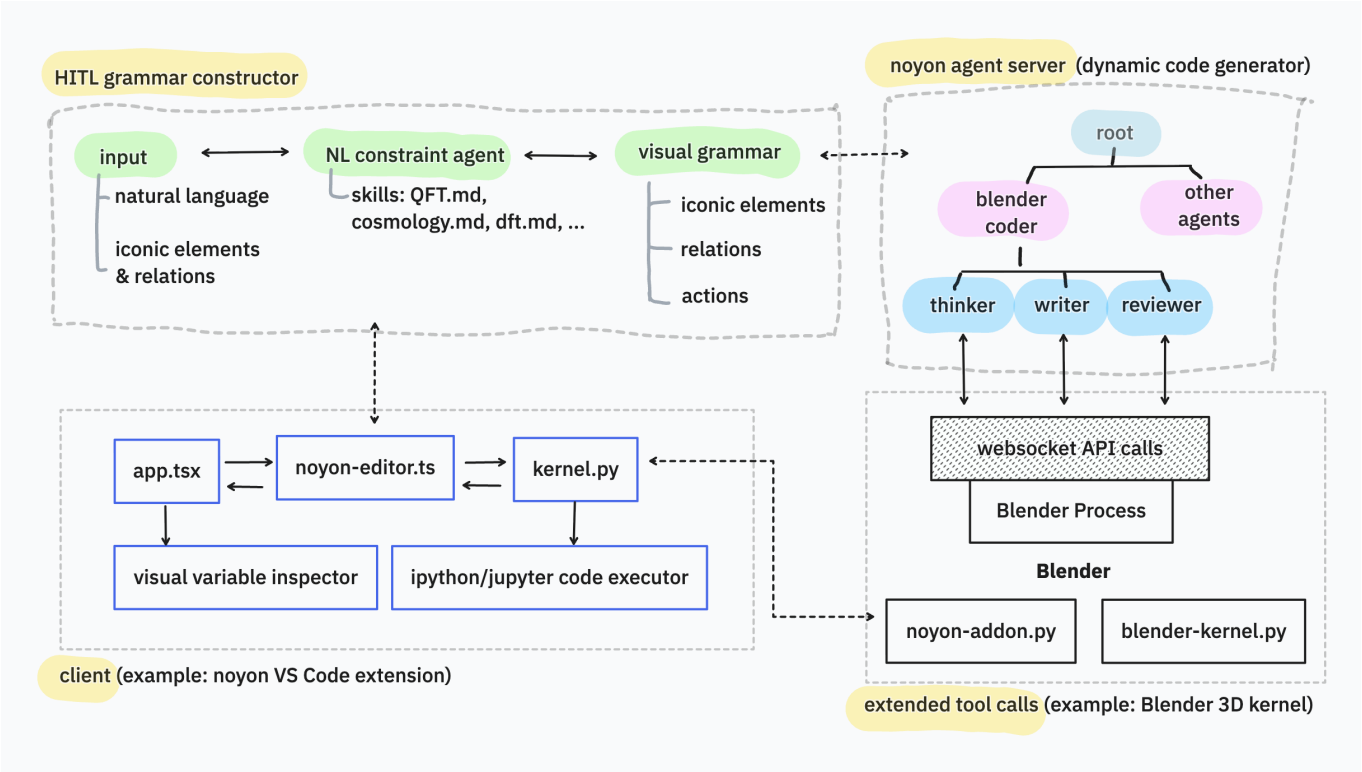


Figure 1: Proposed architecture sketch for Noyon.

Agentic systems utilizing LLMs offer a unique set of solutions to this problem. Agentic systems can be used to a) interpret natural language text to visual grammars that define the elements and interactions for the problem at hand, and b) generate code dynamically (along with a light tool-calling pipeline) to build and deploy the interaction on the fly.

To work intuitively using this system of agents, our thesis is that existing system designs for CAS and mathematical coding also need to evolve, with options for various inputs (e.g., natural language, pen and touch inputs enabling drawing, etc.). These input modalities need to coordinate with the agentic infrastructure seamlessly.

Noyon is a system that combines the agentic infrastructure and input interaction methods to provide a seamless mathematical computing experience. The interaction designs take inspiration from existing research works (as pointed out in later sections). We report the system architecture and the current state of implementation, along with a brief survey of literature.

2. Related Works

Our work draws inspiration from three intersecting research threads: (1) embodied and visual representations for mathematical understanding, (2) AI agents for mathematical and scientific computing, and (3) agentic and embodied interaction paradigms in HCI. We survey each below, situating our proposed architecture at their convergence.

2.1. Embodied Cognition and Visual Grammars for Mathematics

The theoretical foundation for embodied mathematics originates in Lakoff and Núñez's influential thesis that mathematical reasoning is grounded in sensory-motor experience and conceptual metaphor [18]. Their account argues that even abstract constructs—from arithmetic to transfinite numbers—arise through embodied cognitive mechanisms such as image schemas and metaphorical projection. This perspective motivates our architectural emphasis on translating symbolic mathematical abstractions as manipulable, spatially grounded representations.

Within HCI, a growing body of work has explored how visual augmentation can make mathematical notation more accessible and interactive. Head et al. [13] conducted a systematic qualitative analysis of *math augmentation*—the practice of embellishing formulas with color, labels, spacing, and other visual design patterns to improve readability—identifying recurring strategies across over 1,100 examples. Tao et al. [28] introduced *FreeForm*, a projectional editor that synchronizes graphical formula augmentations with underlying LaTeX markup, demonstrating that authors work most effectively when they can

fluidly interleave graphical and symbolic editing across multiple co-present views. Perhaps most directly relevant to our vision is Arawjo et al.'s *Notational Programming* [1], which introduced a system in which handwritten domain-specific notation—demonstrated through quantum circuit diagrams—interoperates with code within Jupyter notebooks. This work is one of the precursors to our architecture: it demonstrates that notation and code can coexist in a shared computational substrate, though it required hand-designed recognizers for each notational domain. One of our goals is to generalize such schemes by using LLM agents to mediate between mathematical text, visual notation, and executable code.

A parallel line of work has pursued interactive and AR-based explorable explanations for STEM content. Chulpongsatorn et al. [5] presented *Augmented Math*, which uses OCR and computer vision to extract formulas from static textbooks and overlay interactive AR animations—including dynamic values, relationship highlights, and step-by-step hints—without requiring programming. Gunturu et al. [9] extended this idea to physics with *Augmented Physics*, using Segment Anything and multimodal LLMs to transform textbook diagrams into embedded interactive simulations spanning optics, circuits, and kinematics. These systems demonstrate the power of making static STEM representations dynamic and manipulable, and they preview the role that LLMs can play in bridging between static notation and interactive behavior. However, their augmentation vocabularies—five techniques in Augmented Math, four strategies in Augmented Physics—remain hand-designed for specific domains. Noyon's architecture aims to address this scalability limitation by proposing an LLM-centric grammar that can extract and represent mathematical relationships across domains without domain-specific augmentation engineering.

Hayatpur et al. [11] offer a complementary perspective with *The Shapes of Abstraction in Data Structure Diagrams*, which surveys 80 programmer-produced diagrams to formalize "abstraction moves"—operations that select, revisualize, simplify, or annotate data structures. Their implementation, *Chisel*, reproduces 78 of 80 surveyed diagrams through a composable language of visual transformations. This work, together with their earlier study of informal programmer diagramming practices [10] and multi-level execution visualization in *CrossCode* [12], demonstrates that humans naturally create layered visual abstractions to reason about complex structures, and that these abstractions can be computationally formalized. Our grammar component draws inspiration from this insight: rather than defining a fixed visual vocabulary, we seek an LLM-driven approach to extracting the *relational structure* of mathematical expressions iteratively, which can then be rendered through multiple visual and embodied modalities.

2.2. AI Agents for Mathematical and Scientific Computing

The use of LLM-based agents for mathematical tasks has been explored from several angles in recent HCI research. A prominent thread examines LLMs as educational agents. Jin et al. [16] introduced *AlgoBo*, an LLM-based teachable agent for programming education that uses a "Reflect-Respond" pipeline to restrain the model's knowledge and encourage learning-by-teaching. This was extended to university-level mathematics and engineering courses with *MatlabTutee* [17], which evaluates how effectively an LLM can feign ignorance to support student teaching. *TutorCraftEase* [30] takes a complementary approach, using LLMs to assist instructors in generating pedagogical questions with proper mathematical rendering. These systems demonstrate the value of LLM agents in educational mathematics and focus on conversational tutoring rather than on the computational and representational workflows central to our architecture.

Closer to our vision of agent-assisted computing are systems that make LLM reasoning processes visible and steerable during analytical work. Xie et al. [32] presented *WaitGPT*, which provides on-the-fly code visualization to let users monitor and steer a conversational LLM agent during data analysis, addressing the transparency problem of black-box agent execution. Freund et al. [7] introduced *Flowco*, a mixed-initiative system that structures data analyses as dataflow graphs with LLM-assisted code generation at each node, making the analytical pipeline explicit and inspectable. Jiang et al. [15] proposed *Graphologue*, which transforms LLM text responses into interactive node-link diagrams for structured exploration, while Suh et al. [27] developed *Sensecape* for multilevel exploration and sensemaking with LLMs. Together, these works establish design patterns for AI-mediated visual sensemaking—making LLM reasoning visible through diagrams, multilevel views, and dataflow representations. Our architecture extends this principle to mathematical domains specifically, where the "reasoning" to be made visible includes symbolic manipulation, numeric computation, and the relationships between them.

The broader landscape of LLM-powered coding assistants provides architectural precedent for our agent design. The rapid evolution of AI coding tools—from code completion to multi-agent systems with tool-calling capabilities—has been documented in recent surveys spanning CHI, UIST, and software engineering venues [3]. Arawjo et al. [2] developed *ChainForge*, an open-source visual toolkit for prompt engineering and LLM hypothesis testing that provides a dataflow-style graphical interface for comparing responses across models and prompt variations, identifying three modes of LLM interaction: opportunistic exploration, limited evaluation, and iterative refinement. Systems like *CoLadder* [25], which enables manipulation of LLM code

generation through multi-level blocks, and *NeuroSync* [33], which modifies LLM understanding directly rather than relying on prompting alone, point toward more nuanced human-agent collaboration in code-centric workflows. Our coding agents for mathematical computing and embodied interactions draw inspirations from these architectural patterns, while focusing on the demands of scientific computing, where correctness, symbolic precision, and visual interpretation must coexist.

2.3. Embodied and Agentic Interaction Paradigms

A third body of work explores how AI agents gain richer capabilities through embodied interaction—whether in virtual environments, augmented reality, or physical space. Chulpongsatorn et al. [4] studied how embodiment and multimodal signaling affect prompting behavior when users interact with a VR-based AI agent (CHI 2025). Through a Wizard of Oz study, they found that an agent exhibiting spatial awareness—turning its head toward referenced objects, highlighting targets—helped users prevent errors 30% of the time and increased satisfaction. This work directly informs our design: for embodied mathematical interactions, the agent must signal its understanding of spatial and structural relationships, not merely respond to textual commands.

At the system level, several recent works demonstrate agentic architectures for AR and spatial computing. Zhu et al. [35] introduced *agentAR*, enabling users to create AR applications through tool-augmented LLM-based autonomous agents that handle 3D content generation and spatial reasoning via natural language. Zhao et al. [34] presented *Guided Reality*, which generates visually-enriched AR task guidance using LLMs and vision models. Qian et al. [24] explored *SHAPE-IT*, which uses LLMs to generate shape-changing behaviors for physical pin displays from text descriptions, bridging generative AI with tangible physical outputs. These works demonstrate that LLM agents can effectively mediate between natural language intent and spatial, embodied output—a capability our architecture leverages for mathematical visualization and interaction.

The challenge of maintaining user agency in agentic workflows has also received attention. Peng et al. [23] presented *Morae*, which proactively pauses UI agents at decision points to preserve user choice, while Epperson et al. [6] developed tools for interactive debugging and steering of multi-agent AI systems. The foundational *Generative Agents* work by Park et al. [22] established architectural patterns—observation, planning, and reflection—for believable agent behavior that have influenced subsequent agentic system design. Our multi-agent architecture for mathematical computing inherits these concerns: the agents that extract mathematical structure, generate visual notation, and orchestrate tool-calling must remain transparent and steerable, particularly when operating on expressions where symbolic correctness is paramount.

2.4. Positioning Our Contribution

Taken together, the prior work reveals a gap at the intersection of these three threads. Embodied and visual approaches to mathematics (Augmented Math, Augmented Physics, FreeForm, Chisel) have demonstrated the value of interactive, spatially grounded representations but rely on hand-crafted augmentation strategies that do not generalize across the breadth of mathematical abstractions. Arawjo et al.'s notational programming [1] showed that handwritten notation can interoperate with typed code in computational notebooks, but required domain-specific recognizers rather than leveraging LLM agents for generalization. AI agents for computation (WaitGPT, Flowco, ChainForge, coding assistants) have shown how LLMs can participate in analytical and pipeline-construction workflows but have not specifically addressed mathematical representation or embodied interaction. Embodied agent architectures (agentAR, Prompting an Embodied AI Agent, SHAPE-IT) have explored spatial grounding for AI but have not been applied to the particular challenges of mathematical and scientific computing. Our proposed architecture bridges these gaps by introducing an LLM-centric grammar for extracting mathematical relationships, coding agents for embodied interactions, and an enhanced tool-calling orchestration layer—combining the representational richness of visual grammar research with the scalability of agentic AI.

Beyond academic contribution, a major direction for the Noyon project is to establish an open source project for agent-assisted and embodied interactions based mathematical and scientific computing interface. Scaling and combining the inspirations of some of the previous works to verifiable and reliable software is the goal of the Noyon project.

3. Noyon Architecture

3.1. Core Modules

The system architecture has four main modules (figure 1).

- Client: a client is the visible UI of the system. The user mostly interacts with the client. It's primarily hosted in an IDE (or could be a standalone IDE by itself). Other than code and plots (the standard cells in a notebook), it has the capacity to take pen input (via a newly introduced drawing cell) for drawing strokes, and a Natural Language Input (NLI) cell as an alternative to the code cell, which can output plots and the results of computations dictated by the user in the input cell.

- HITL Grammar Constructor: via the drawn strokes and/or natural language input, this is an agent that constructs and constraints the input to a set of iconic elements, relations, and actions. The skills used by this agent are specific to different fields in math, science, and engineering. Given a piece of text, it orchestrates which skills should be used to generate the iconic elements and relations, in an iterative manner.
- Noyon Agent Server: this is a growing collection of agents that dynamically generate code to supercharge the tools available to the coding assistant and natural language input cell interpreter.
- Extended Tool Calls: the tools available to agents are currently pretty static in nature: once the tools are developed and deployed, they are deterministic in nature and cannot be changed on the fly to the user's input. We introduce an enhanced capability where the agent server generates code on the fly and executes the code inside safe sandboxes.

3.1.1 Inter-module Communication

Noyon has a mix of A2A (agent-to-agent) and custom system call protocols that optimize for context length (under development), which is a practical challenge for, for example, heavy scene descriptions generated in Blender that needs to be fed back into the reviewer sub-agent under the Blender Coder agent.

3.2. Extended Tool Calls

In this paper, we use Blender as an example of utilizing mature software stacks as tools available to Noyon agents. Any scientific software with SDKs or system call options exposed to developers can be integrated within the Noyon architecture.

In practice, an end-to-end example could look like this: the user asks in an NLI cell to create a 3D shape, subdivide the mesh, deform one corner of it, and then finally run a heat simulation on the mesh and visualize the temperature reached in each face of the mesh.

Using the current workflows typically available to a soft-matter robotics designer, this problem statement would require running multiple workflows using multiple software—Blender for mesh design, editing and deformation, COMSOL for running the heat simulation, and then combining outputs from each software in a particular sequence, load the output data in a notebook and analyze/visualize the final results, and iterate over the whole process.

In the proposed setup, the effort should be reduced significantly. The visual grammar agent could take the user request in plain text (or a mix of visual input and text), convert those into a set of instructions for the *Blender Coder* agent, which utilizes its sub-agents to dynamically generate Blender scripts that can be run inside a Blender sandbox. The output of this extended tool call (which utilizes Blender's functionalities for mesh creation, editing, and deformation) is then fed back into the Client.

Next, either using the COMSOL SDK or by generating code to be executed in a notebook cell, the agent server generates a heat simulation and it's then run in the tool call sandbox.

The result is then visualized via the Blender kernel, relying on code generated by the agent server once again.

This dynamic code generation-based tool calls enable us to extend Noyon's capabilities significantly, and delegates multiple manual workflows to a system of agents. While reliability of the generated code has not been studied yet, based on preliminary results, we are encouraged to pursue this direction.

3.3. Input Capabilities

Noyon's "frontend" is hosted on VS Code as an extension. However, with some changes in the code it can be turned into a standalone IDE if required. A user can create drawing and NLI cells anywhere in the notebook, in between code, markdown, and plot outputs.

- Drawing Cells: Strokes drawn on the drawing cells are observable and manipulable via a visual variable inspector. More importantly, they are reference-able from the standard code cells, utilizing the strokes directly inside the code. The strokes information can also be passed to the NL constraint agent to aid in constructing the relations between the iconic elements.
- Natural Language Input (NLI) Cells: users can input natural language, refer to code cells, drawn objects, 3D models, etc. from the input text, and request the agent server to manipulate, change, or edit the outcomes.

3.4. Examples

- Textbook Explanation / Tutorial Video Generation: this is actively under development. It utilizes the end-to-end pipeline defined in this architecture. Starting from a textbook section, the visual grammar agent generates a set of iconic elements and instructions for visualization that are passed to the Noyon agent server. Code generated by the Blender Coder agent is then executed in the Blender sandbox, and sent back to the Client.
- Logo-like stroke manipulation via code: using the visual grammar agent and drawing cells in the Client,

we can write code to manipulate strokes in the drawing cells, creating a Logo-like environment using any programming language allowed by the notebook.

- Creating and modifying 3D models: example described in section 3.2.
- Running Complex Simulations: example described in section 3.2
- Data Analysis and Visualization: Unlike some current agentic workflows for data analysis and visualization, Noyon can utilize existing data analysis tools like R to execute data workflows using dynamically generated R code.

4. Implementation: Current State and Challenges

The Noyon team has built the first version for the Client, Extended Tool Call sandbox, and the Noyon agent server. The Visual Grammar Constructor is in active development.

4.1. Current Challenges

- Visual Grammar Agent: our initial experiments with a set of agent skills yielded some successful results. However, we need to figure out how a visual grammar agent can generalize to a wide variety of abstractions. We believe this is where LLM-based agents can add a lot of value, moving our community beyond handcrafted and/or limited design study based grammars to a practical, scalable alternative.
- Verification and Reliability of Extended Tool Calls: results produced by our pipeline have worked well in some settings (such as the mesh example, or logo-like manipulation of strokes in the drawing cells), but robust verification methods need to be devised for this particular architecture.
- Growing the Tools Collection: for an open source project, this is crucial. The Noyon team invites interested communities to help us create sandboxes for other computational and data tools.

References

[1] Ian Arawjo, Anthony DeArmas, Michael Roberts, Shrutarshi Basu, and Tapan Parikh. Notational programming for notebook environments: A case study with quantum circuits. In *Proceedings of the 35th Annual ACM Symposium on User Interface Software and Technology*, UIST '22, New York, NY, USA, 2022. ACM.

[2] Ian Arawjo, Chelse Swoopes, Priyan Vaithilingam, Martin Wattenberg, and Elena L. Glassman. Chainforge: A visual toolkit for prompt engineering and llm hypothesis testing. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*, CHI '24, pages 1–18, New York, NY, USA, 2024. ACM. Best Paper Honorable Mention.

[3] Nghi D. Q. Bui. Building effective ai coding agents for the terminal: Scaffolding, harness, context engineering, and lessons learned, 2026.

[4] Neil Chulpongsatorn et al. Prompting an embodied ai agent: How embodiment and multimodal signaling affects prompting behaviour. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, CHI '25, New York, NY, USA, 2025. ACM.

[5] Neil Chulpongsatorn, Mille Skovhus Lunding, Nishan Soni, and Ryo Suzuki. Augmented math: Authoring ar-based explorable explanations by augmenting static math textbooks. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, UIST '23, New York, NY, USA, 2023. ACM.

[6] Will Epperson, Gagan Bansal, Victor Dibia, Adam Fourney, Jack Gerrits, Erkang Zhu, and Saleema Amershi. Interactive debugging and steering of multi-agent ai systems. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, CHI '25, New York, NY, USA, 2025. ACM.

[7] Stephen Freund, Brooke Simon, Emery Berger, and Eunice Jun. Flowco: Mixed-initiative authoring of reliable end-to-end data analyses via dataflow graphs and llms. In *Proceedings of the 38th Annual ACM Symposium on User Interface Software and Technology*, UIST '25, New York, NY, USA, 2025. ACM.

[8] Michael Gray. Sage: A new mathematics software system. 10(6):72–75, 2008. http://ieeexplore.ieee.org/xpls/abs_all.jsp?arnumber=4653208.

[9] Aditya Gunturu, Yi Wen, Nandi Zhang, Jarin Thundathil, Rubaiat Habib Kazi, and Ryo Suzuki. Augmented physics: Creating interactive and embedded physics simulations from static textbook diagrams. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology*, UIST '24, New York, NY, USA, 2024. ACM. Best Paper Award.

[10] Devamardeep Hayatpur, Brian Hempel, Kathy Chen, William Duan, Philip Guo, and Haijun Xia. Taking ascii drawings seriously: How programmers diagram code. In *Proceedings of the 2024 CHI Conference on*

Human Factors in Computing Systems, CHI '24, New York, NY, USA, 2024. ACM.

[11] Devamardeep Hayatpur, Brian Hempel, Richard Lin, and Haijun Xia. The shapes of abstraction in data structure diagrams. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, CHI '25, New York, NY, USA, 2025. ACM.

[12] Devamardeep Hayatpur, Daniel Wigdor, and Haijun Xia. Crosscode: Multi-level visualization of program execution. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, CHI '23, New York, NY, USA, 2023. ACM.

[13] Andrew Head, Amber Xie, and Marti A. Hearst. Math augmentation: How authors enhance the readability of formulas using novel visual design practices. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems*, CHI '22, New York, NY, USA, 2022. ACM.

[14] Wolfram Research, Inc. Mathematica, Version 12.0, 2019. Champaign, IL, 2019.

[15] Peiling Jiang, Jude Rayan, Steven P. Dow, and Haijun Xia. Graphologue: Exploring large language model responses with interactive diagrams. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, UIST '23, New York, NY, USA, 2023. ACM.

[16] Hyoungwook Jin, Seonghee Lee, Hyungyu Shin, and Juho Kim. Teach ai how to code: Using large language models as teachable agents for programming education. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*, CHI '24, New York, NY, USA, 2024. ACM.

[17] Kantwon Rogers, Michael Davis, Mallesh Maharana, Pete Etheredge, Sonia Chernova. Playing dumb to get smart: Creating and evaluating an llm-based teachable agent. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, CHI '25, New York, NY, USA, 2025. ACM.

[18] George Lakoff and Rafael E. Núñez. *Where Mathematics Comes From: How the Embodied Mind Brings Mathematics into Being*. Basic Books, New York, NY, USA, 2000.

[19] Maplesoft, a division of Waterloo Maple Inc.. Maple, 2019.

[20] MATLAB. *version 7.10.0 (R2010a)*. The MathWorks Inc., Natick, Massachusetts, 2010.

[21] 2020. System for the manipulation of symbolic and numerical expressions. Includes a collection of special functions. Utilizes exact fractions, arbitrary precision integers, and arbitrary precision floating point numbers.

[22] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, UIST '23, pages 1–22, New York, NY, USA, 2023. ACM.

[23] Yi-Hao Peng, Ding Li, Jeffrey Bigham, and Amy Pavel. Morae: Proactively pausing ui agents for user choices. In *Proceedings of the 38th Annual ACM Symposium on User Interface Software and Technology*, UIST '25, New York, NY, USA, 2025. ACM.

[24] Michael Qian, Chenfeng Gao, Anup Sathya, Ryo Suzuki, and Ken Nakagaki. Shape-it: Exploring text-to-shape-display for generative shape-changing behaviors with llms. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology*, UIST '24, New York, NY, USA, 2024. ACM.

[25] Ryan Yen, Jiawen Zhu, Sangho Suh, Haijun Xia, Jian Zhao. Coladder: Manipulating code generation via multi-level blocks. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology*, UIST '24, New York, NY, USA, 2024. ACM.

[26] Nazmus Saquib, Rubaiat Habib Kazi, Li-Yi Wei, Gloria Mark, and Deb Roy. Constructing embodied algebra by sketching. In *CHI' 21*, 2021.

[27] Sangho Suh, Bryan Min, Srishti Palani, and Haijun Xia. Sensecape: Enabling multilevel exploration and sensemaking with large language models. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, UIST '23, New York, NY, USA, 2023. ACM.

[28] Jeffrey Tao, Litao Yan, Jessica Shi, Mia Ginsberg, and Andrew Head. Freeform: Flexibly augmenting formulas with synchronized markup and graphical edits. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, CHI '25, New York, NY, USA, 2025. ACM.

[29] Bret Victor. Kill math, 2011.

[30] Wenhui Kang, Lin Zhang, Xiaolan Peng, Hao Zhang, Anchi Li, Mengyao Wang, Jin Huang, Feng Tian, Guozhong Dai. Tutorcraftease: Enhancing pedagogical question creation with large language models. In

Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems, CHI '25, New York, NY, USA, 2025. ACM.

[31] Stephen Wolfram. Finally we may have a path to the fundamental theory of physics, and it's beautiful, 2020.

[32] Liwenhan Xie, Chengbo Zheng, Haijun Xia, Huamin Qu, and Chen Zhu-Tian. Waitgpt: Monitoring and steering conversational llm agent in data analysis with on-the-fly code visualization. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology*, UIST '24, New York, NY, USA, 2024. ACM.

[33] Wenshuo Zhang et al. Neurosync: Intent-aware code-based problem solving via direct llm understanding modification. In *Proceedings of the 38th Annual ACM Symposium on User Interface Software and Technology*, UIST '25, New York, NY, USA, 2025. ACM.

[34] Ada Zhao, Aditya Gunturu, Ellen Yi-Luen Do, and Ryo Suzuki. Guided reality: Generating visually-enriched ar task guidance with llms and vision models. In *Proceedings of the 38th Annual ACM Symposium on User Interface Software and Technology*, UIST '25, New York, NY, USA, 2025. ACM.

[35] Chenfei Zhu, Shao-Kang Hsia, Xiyun Hu, Ziyi Liu, Jingyu Shi, and Karthik Ramani. agentar: Creating augmented reality applications with tool-augmented llm-based autonomous agents. In *Proceedings of the 38th Annual ACM Symposium on User Interface Software and Technology*, UIST '25, New York, NY, USA, 2025. ACM.